

Relatorio Tecnico

NEON SIMD e Macros GAS

Implementacao completa dos 12 exercicios praticos do TP3, cobrindo manipulacao de registradores vetoriais ARM64, operacoes SIMD com inteiros e ponto flutuante, shuffle de bytes, reducao controlada e macros GAS basicas e parametrizadas. A validacao foi feita por execucao AArch64 em `qemu-aarch64`, desassemblagem com `objdump` e amostras de registradores por `gdb-multiarch`.

ALUNO Renato	AMBIENTE DE VALIDACAO aarch64-linux-gnu-gcc + qemu-aarch64 + gdb-multiarch
DISCIPLINA DR4 - Processamento Otimizado em Assembly ARM 64-bits	ALVO PREVISTO Raspberry Pi Zero 2W / Linux ARM64
ENTREGA TP3	ARTEFATOS Codigo Assembly, suite de testes, evidencias textuais e relatorio

1. Objetivo e escopo

O objetivo deste TP3 foi implementar doze exercicios em Assembly ARM64 com foco em NEON SIMD e automacao por macros do GAS, mantendo cada solucao pequena, reproduzivel e diretamente validavel por testes automatizados.

As rotinas foram expostas para um harness em C que prepara vetores de entrada, chama o codigo Assembly e compara todas as saidas com valores esperados calculados independentemente. Isso permite validar o comportamento funcional no QEMU e, ao mesmo tempo, preservar labels internos para inspecao por GDB.

Decisao de ambiente: o enunciado permite executar no Raspberry Pi Zero 2W ou validar com QEMU. A entrega gera um executavel ELF 64-bit ARM aarch64 estatico, com simbolos de debug, compativel com Linux ARM64 e testado localmente por emulacao.

Pontos relevantes da entrega

- As 12 rotinas pedidas no enunciado foram implementadas em Assembly ARM64.
- Os exercicios 1 a 7 validam manipulacao direta de lanes e registradores vetoriais.
- Os exercicios 8 a 10 validam paralelismo SIMD para inteiros e float32.
- Os exercicios 11 e 12 validam macros GAS basicas e parametrizadas por objdump.
- As evidencias foram registradas em texto: QEMU, GDB, objdump e metadados do binario.

2. Organizacao do projeto

O projeto segue a mesma organizacao utilizada no TP2: implementacao isolada em src, declaracoes em include, testes em tests, evidencias em evidencias e relatorio final em relatorios.

- Assembly ARM64
- NEON SIMD
- Macros GAS
- QEMU user mode
- GDB remoto
- Relatorio HTML + PDF

Arquivo / pasta	Funcao
src/tp3_exercises.S	Implementa as 12 rotinas, a macro basica, a macro parametrizada e o padrao de shuffle.
include/tp3.h	Declara as interfaces usadas pelo harness de validacao.
tests/test_tp3.c	Executa a validacao automatizada com comparacao lane a lane e byte a byte.
scripts/capture_gdb_registers.sh	Abre QEMU em modo GDB remoto e captura alguns registradores vetoriais em labels internos.
Makefile	Coordena compilacao, execucao em QEMU, objdump, GDB e geracao do PDF.
evidencias/	Contem saidas de teste, formato do binario, desassemblagem e captura GDB.

Fluxo de validacao

- ### 1. Implementacao

Rotinas em Assembly ARM64 usam NEON, convencao AAPCS64 e buffers C para observabilidade.
- ### 2. Cross-build

O projeto e compilado por aarch64-linux-gnu-gcc e ligado estaticamente.
- ### 3. Execucao

O binario AArch64 roda em qemu-aarch64 e retorna erro em qualquer falha.
- ### 4. Evidencias

make evidence salva QEMU, ELF e objdump; make gdb-evidence salva GDB.

```
make
make test
make evidence
make gdb-evidence
make report
```

3. Implementacao dos exercicios

A tabela resume a solucao tecnica adotada. As funcoes foram escritas para expor o estado final em memoria ou em registrador geral, facilitando validacao automatica e inspecao manual.

Ex.	Implementacao	Observacao tecnica
1	mov v0.s[0], w0 apos carregar um vetor inicial.	Atualiza somente os 32 bits baixos e preserva os demais lanes.
2	mov v0.d[0], x0 para inserir um escalar de 64 bits.	A metade alta de 64 bits do registrador vetorial permanece intacta.
3	eor v0.16b, v0.16b, v0.16b.	Zera todos os 128 bits de V0 de forma deterministica.
4	movi v0.16b, #0xff.	Cria mascara com todos os bits setados.
5	tbl v2.16b, {v0.16b}, v1.16b com tabela de indices propria.	Padrao aplicado: 15,14,13,12,8,9,10,11,3,2,1,0,4,5,6,7.
6	mov v0.s[2], w0.	Insera uma word no lane especifico s[2].
7	shl v0.2d, v0.2d, #5.	Desloca os dois lanes de 64 bits sem propagacao entre lanes.
8	add v2.16b, v0.16b, v1.16b.	Executa 16 somas inteiras de 8 bits em paralelo, com wraparound natural.
9	umull, umull2, add e addv.	Multiplica oito pares u16 e reduz para w0.
10	fadd v2.4s, v0.4s, v1.4s.	Valida soma SIMD em quatro valores float32.
11	Macro INIT_VEC_ZERO reg expandida em dois registradores.	O objdump mostra duas expansoes independentes com eor.
12	Macro SIMD_ADD16_U8_BLOCK parametriza registradores, bases e offset.	A macro e invocada para offsets 0 e 16.

Macros implementadas

Macro basica

INIT_VEC_ZERO reg recebe o registrador vetorial e gera uma operacao eor reg.16b, reg.16b, reg.16b. Ela foi usada sobre v0 e v1.

Macro parametrizada

SIMD_ADD16_U8_BLOCK recebe registradores vetoriais de destino/entradas, registradores-base de memoria e offset. Assim, a mesma sequencia load-operate-store e gerada para multiplos blocos.

4. Validacao e evidencias

A validacao principal foi automatizada no harness C. Cada exercicio compara todos os bytes, lanes ou escalares relevantes e o programa retorna codigo de erro se qualquer comparacao falhar. O log completo esta em evidencias/qemu_test_output.txt.

Confirmacao do binario gerado

```
build/tp3_neon_macros: ELF 64-bit LSB executable, ARM aarch64, version 1 (GNU/Linux),
statically linked, for GNU/Linux 3.7.0, with debug_info, not stripped
```

Resumo da suite

Grupo	Validacao executada	Resultado
Exercicios 1 a 7	Insercao de lanes, zeragem, preenchimento, shuffle e shift por lane.	Todos aprovados
Exercicios 8 a 10	Soma u8x16, dot product u16x8 e soma float32x4.	Todos aprovados
Exercicios 11 e 12	Execucao funcional das macros e evidencia de expansao por objdump.	Todos aprovados

Trecho do output de execucao

```
Validacao automatizada do TP3 - NEON SIMD e macros GAS
Execucao cruzada: binario AArch64 rodando em qemu-aarch64

Exercicio 1 - Wn para lane baixa de Vd
[ OK ] lane s[0] atualizada e demais lanes preservados: [0x12345678, 0xbbbbbbbb, 0xcccccccc, 0xdddddddd]

Exercicio 8 - Soma SIMD de 16 bytes
[ OK ] 16 somas u8 processadas em paralelo: f1 f7 fd 03 09 0f 15 1b 21 27 2d 33 39 3f 45 4b

Exercicio 12 - Macro parametrizada para dois blocos SIMD
[ OK ] dois blocos de 16 bytes gerados por macro: a0 9e 9c 9a 98 96 94 92 90 8e 8c 8a 88 86 84 82 80 7e 7c 7a 78 76 74 72 70 6e 6c 6a 68 66 64 62

Resultado final: todos os 12 exercicios passaram nos testes.
```

Evidencia de objdump para macros

```
ex11_macro_zero_twice:
    movi    v0.16b, #0x5a
    eor     v0.16b, v0.16b, v0.16b
    movi    v1.16b, #0xa5
    eor     v1.16b, v1.16b, v1.16b

ex12_macro_add_two_blocks:
    ld1     {v0.16b}, [x9]
    ld1     {v1.16b}, [x10]
    add     v2.16b, v0.16b, v1.16b
    st1     {v2.16b}, [x11]
    ld1     {v3.16b}, [x9]
    ld1     {v4.16b}, [x10]
    add     v5.16b, v3.16b, v4.16b
    st1     {v5.16b}, [x11]
```

Amostras de GDB

```
[ex1_after_insert] v0.s = {0x12345678, 0xbbbbbbbb, 0xcccccccc, 0xdddddddd}
[ex2_after_insert] v0.d = {0x0123456789abcdef, 0xbbbbbbbbbbbbbbbb}
[ex5_after_shuffle] v2.b = {0xff, 0xee, 0xdd, 0xcc, 0x88, 0x99, 0xaa, 0xbb, 0x33, 0x22, 0x11, 0x00, 0x44, 0x55, 0x66, 0x77}
[ex9_after_reduce] w0 = 0x00002b3e
[ex11_after_first_zero] v0.b = {0x00 repetido 16 vezes}
```

As saidas completas foram preservadas em evidencias/qemu_test_output.txt, evidencias/objdump_tp3_exercises.txt e evidencias/gdb_registers.txt.

5. Conclusao

A entrega atende ao enunciado do TP3: os 12 exercicios foram implementados em Assembly ARM64 usando NEON SIMD e macros GAS, organizados em um projeto reproduzivel, compilados para AArch64 e validados por execucao cruzada em QEMU.

Alem da validacao funcional, a entrega inclui evidencias por objdump para expansao das macros e capturas GDB de registradores vetoriais em pontos internos da execucao. A suite automatizada terminou sem falhas.